ЧАСТЬ 1. ОСНОВЫ PYTHON

1.2. Ввод, вывод, f-строки

К концу занятия ты сможешь:

- получать ответы пользователя через input()
- превращать ввод в числа и проверять, что ввели цифры
- красиво выводить числа через f-строки
- управлять выводом print с помощью sep и end

Разминка: вспомним тему 1.1

1. Какой тип у каждого значения: `15`, `15.0`, `"15"`, `True`?

2. Как узнать тип значения, не угадывая? Какую функцию для этого вызвать?

3. Было `score = 10`, потом написали `score = 25`. Что теперь хранится в score и куда делось 10?

Вот что у тебя получится сегодня

Программа «Чек для друзей» спрашивает, где вы ели, сколько стоил счёт и сколько вас было.

Потом печатает аккуратный чек: суммы с копейками и делением на всех.

Пока твои программы не умеют ничего спрашивать. К концу занятия научатся.

Вывод в консоли

```
Где ели: Пицца на углу   <- ввёл пользователь
Сумма счёта: 3470   <- ввёл пользователь
Сколько человек: 4   <- ввёл пользователь
--------------------------
Кафе: Пицца на углу
Счёт: 3,470.00 руб.
Каждый платит: 867.50 руб.
```

Стрелка — пометка для тебя, в программе её нет

План занятия

УРОК 1

Программа спрашивает и считает

input(), int(), float(), str(), проверка .isdigit()

Практика: «Делим счёт в кафе»

УРОК 2

Красивый вывод через f-строки

f"...{...}...", знаки после точки, разделитель тысяч

Практика: «Статистика канала»

УРОК 3

Параметры print: sep и end

Практика: «Табло матча»

Мини-проект: «Касса кинотеатра»

Вспомним: типы данных (тема 1.1)

Тип	Пример	Что это
int	15	целое число
float	1.75	дробное число, всегда через точку
str	"Лиза", "15"	текст в кавычках, даже если внутри цифры
bool	True, False	да или нет

Переменная — имя, подписанное на значение. type(значение) показывает тип.

Сегодня важнее всего разница между **15** и **"15"**: число и текст.

УРОК 1 • БЛОК 1 ИЗ 3

Программа спрашивает и считает

В этом блоке ты научишься:

- получать ответ пользователя через input()
- превращать текст в число через int() и float(), число в текст через str()
- проверять, что введены только цифры, через .isdigit()

input(): программа задаёт вопрос

Что это. input() показывает вопрос, останавливает программу и ждёт, пока человек напечатает ответ и нажмёт Enter.

Похоже на анкету: вопрос, пустое поле, ответ.

Зачем. Без input() программа всегда здоровается с одним и тем же человеком. Чтобы сменить имя, придётся править код.

Без input(): имя зашито в код

main.py

```
nickname = "Лиза"
print("Привет,", nickname)
```

Вывод в консоли

```
Привет, Лиза
```

Запусти хоть сто раз — всегда «Лиза».

input(): как записывается

`nickname =`	`input(`	`"Введи ник: "`	`)`
куда положить ответ	спросить и ждать	что увидит человек	

main.py

```
nickname = input("Введи ник: ")
print("Привет,", nickname)
```

Вывод в консоли

```
Введи ник: Shadow_Fox    <- ввёл пользователь
Привет, Shadow_Fox
```

Стрелка — пометка для тебя, в программе её нет

Разбор. Строка 1 показывает «Введи ник: » и ждёт Enter, ответ попадает в nickname. Пробел после двоеточия — чтобы ответ не прилип к вопросу.

Правило: input() ждёт ответа и отдаёт то, что напечатал человек.

Ловушка: ввод всегда строка

main.py

```
first = input("Первое число: ")
second = input("Второе число: ")
print("Сумма:", first + second)
print(type(first))
```

Вывод в консоли

```
Первое число: 2   <- ввёл пользователь
Второе число: 2   <- ввёл пользователь
Сумма: 22
<class 'str'>
```

Стрелка — пометка для тебя, в программе её нет

> **Почему 22?** input() всегда отдаёт строку, даже если ввели цифры. "2" + "2" — это склейка текста, а не сложение.
>
> **Как правильно:** превратить строку в число до того, как считать. Как это сделать — на следующем слайде.

int() и float(): текст превращается в число

Что это. int() делает из строки целое число, float() — дробное. Как переводчик с «языка текста» на «язык чисел». **Зачем:** чтобы с ответом можно было считать.

main.py

```
age = int(input("Сколько тебе лет: "))
print("Через год тебе будет", age + 1)
height = float(input("Рост в метрах: "))
print("Рост в сантиметрах:", height * 100)
```

Вывод в консоли

```
Сколько тебе лет: 15   <- ввёл пользователь
Через год тебе будет 16
Рост в метрах: 1.75   <- ввёл пользователь
Рост в сантиметрах: 175.0
```

Стрелка — пометка для тебя, в программе её нет

Разбор. Python выполняет изнутри наружу: сначала input() получает строку "15", потом int() превращает её в число 15. Граница: float() понимает только точку — "1.75", а не "1,75".

Правило: нужно считать с вводом — оберни input() в int() или float().

Разбор: от ответа до числа

main.py

```
text = input("Сколько у тебя монет: ")
coins = int(text)
print(type(text), type(coins))
print("Монет после бонуса:", coins + 50)
```

Вывод в консоли

```
Сколько у тебя монет: 120   <- ввёл пользователь
<class 'str'> <class 'int'>
Монет после бонуса: 170
```

Стрелка — пометка для тебя, в программе её нет



Превращение не меняет text: там по-прежнему строка "120". Число лежит в новой переменной coins.

Правило: int() возвращает новое значение — сохрани его в переменную, иначе оно потеряется.

str(): число превращается в текст

Что это. str() делает из числа строку. **Зачем:** через + можно склеить только строку со строкой. Строка + число — ошибка.

main.py · Было: ошибка

```
score = 150
print("Счёт: " + score)
```

Вывод в консоли

```
Traceback (most recent call last):
  File ".../main.py", line 2, in <module>
    print("Счёт: " + score)
          ~~~~~~~~~~^~~~~~~
TypeError: can only concatenate str (not "int") to str
```

main.py · Стало: работает

```
score = 150
print("Счёт: " + str(score))
```

Вывод в консоли

```
Счёт: 150
```

Красный текст — **traceback**, сообщение об ошибке: Python остановился и объясняет почему. Через запятую str() не нужен: print("Счёт:", score) работает.

Правило: склеиваешь через + — число сначала оберни в str().

Ловушка: int(input()) падает на букве

```
Вывод в консоли · программа «Через год тебе будет…»

Сколько тебе лет: пятнадцать   <- ввёл пользователь
Traceback (most recent call last):
  File ".../main.py", line 1, in <module>
    age = int(input("Сколько тебе лет: "))
          ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'пятнадцать'
```

Читаем traceback снизу вверх:

1. Последняя строка — тип ошибки и причина. ValueError: «неподходящее значение», 'пятнадцать' не превратить в int.
2. Выше — строка кода, где упало. Значки ~~~^^^ показывают место.
3. Ещё выше — файл и номер строки: line 1.

float("1,75") падает так же: ValueError: could not convert string to float: '1,75'

Место для скриншота: эта же ошибка в терминале VS Code

.isdigit(): в строке только цифры?

Что это. .isdigit() проверяет, что в строке только цифры, и отвечает True или False. Запись через точку — метод строки: пока просто запомни её, подробно — в теме 2.1.

Зачем: узнать заранее, сработает ли int(). Что делать при False — в теме 1.4.

main.py

```
text = input("Введи возраст: ")
print("Это целое число?", text.isdigit())
```

Запуск 1

```
Введи возраст: 15   <- ввёл пользователь
Это целое число? True
```

Запуск 2

```
Введи возраст: пятнадцать   <- ввёл пользователь
Это целое число? False
```

Стрелка — пометка для тебя, в программе её нет

Границы: что считается не цифрой

Запись	Ответ	Почему
"15".isdigit()	True	только цифры
"-5".isdigit()	False	минус — не цифра
"3.5".isdigit()	False	точка — не цифра
"".isdigit()	False	пустая строка
" 7".isdigit()	False	пробел — не цифра

Вопросы по блоку 1 • часть 1

1. Почему программа из ловушки вывела «Сумма: 22», а не 4? Что нужно поменять в коде, чтобы получилось 4?

2. Что выведет эта программа, если ввести 100? Почему?

```
print(type(input()))
```

Вопросы по блоку 1 • часть 2

3. Найди ошибку. Пользователь ввёл «15 лет». Что случится? Что будет написано в последней строке traceback?

```
age = int(input("Возраст: "))
```

4. Что вернёт "3.5".isdigit()? Значит ли это, что float("3.5") тоже не сработает?

Задание: «Делим счёт в кафе»

- Программа спрашивает сумму счёта (она может быть с копейками) и сколько вас человек.
- Выводит, сколько платит каждый.
- Потом «сломай программу»: вместо числа людей введи слово. Прочитай traceback снизу вверх и найди строку, где всё упало.

Подсказка: сумма — через float(), число людей — через int(). Вывод пока через запятые в print.

★ **Успел раньше:** добавь 10% чаевых и выведи счёт с чаевыми и долю каждого.

Застрял — 15 минут ищешь сам, потом спрашиваешь: что делал, что ожидал, что получил.

Что должно получиться

```
Обычный запуск
Сумма счёта: 2450    <- ввёл пользователь
Сколько человек: 4    <- ввёл пользователь
Каждый платит: 612.5 руб.
```

```
«Сломай программу»: конец traceback
Сколько человек: четыре    <- ввёл пользователь
ValueError: invalid literal for int() with base 10: 'четыре'
```

Стрелка — пометка для тебя, в программе её нет

Проверь себя

- При вводе 2450 и 4 программа пишет «Каждый платит: 612.5 руб.»
- При вводе «четыре» в последней строке traceback — ValueError, и ты можешь назвать строку кода, где упало.
- Ты можешь объяснить каждую строку своего кода.

★★ «Фокус с числом»

Программа просит загадать число и показывает каждый шаг: умножить на 2, прибавить 10, разделить на 2, вычесть загаданное. Что бы ни загадали, выходит 5.

Твоя задача: повтори этот фокус, а потом придумай свой — с другими шагами, но тоже с одинаковым ответом для любого числа. Проверь его на трёх разных числах.

Вывод в консоли

```
Загадай число: 37   <- ввёл пользователь
Умножаю на 2: 74
Прибавляю 10: 84
Делю на 2: 42.0
Вычитаю загаданное: 5.0
```

Стрелка — пометка для тебя, в программе её нет

УРОК 2 • БЛОК 2 ИЗ 3

Красивый вывод через f-строки

В этом блоке ты научишься:

- вставлять переменные и расчёты прямо в текст через f-строки
- выводить ровно 2 знака после точки: {price:.2f}
- разделять большие числа на тысячи: {views:,}

f-строки: зачем они нужны

Что это. f-строка — строка-шаблон: на место {имени} Python сам подставит значение переменной. Как бланк с пропусками, которые заполняются сами.

main.py · Было: + и запятые

```
name = "Лиза"
level = 7
print("Игрок " + name + " достиг уровня " + str(level) + ".")
print("Игрок", name, "достиг уровня", level, ".")
```

Вывод в консоли

```
Игрок Лиза достиг уровня 7.
Игрок Лиза достиг уровня 7 .
```

main.py · Стало: f-строка

```
name = "Лиза"
level = 7
print(f"Игрок {name} достиг уровня {level}.")
```

Вывод в консоли

```
Игрок Лиза достиг уровня 7.
```

Зачем. С + нужны str() и пробелы вручную, запятые ставят лишний пробел (видишь «7 .»?). f-строка короче, и текст выглядит так, как напечатается.

f-строка: как записывается

`f"`	`Игрок`	`{name}`	`в игре"`
буква f прямо перед кавычкой	обычный текст	сюда подставится значение	снова текст

```
main.py
player = input("Ник: ")
points = int(input("Очки: "))
print(f"{player} набрал {points} очков")
```

```
Вывод в консоли
Ник: Max_Pro    <- ввёл пользователь
Очки: 340    <- ввёл пользователь
Max_Pro набрал 340 очков
```

Стрелка — пометка для тебя, в программе её нет

Разбор. Python берёт значения player и points и вставляет их в текст. points — число, но str() не нужен: f-строка сама превращает значение в текст.

Правило: f перед кавычкой, переменная — в фигурных скобках.

В фигурных скобках можно считать

В {} можно писать не только имя переменной, но и расчёт. Python сначала посчитает, потом подставит результат в текст.

main.py

```
price = 350
count = 3
print(f"{count} билета по {price} руб.")
print(f"Всего: {price * count} руб.")
print(f"Со скидкой 100 руб.: {price * count - 100} руб.")
```

Вывод в консоли

```
3 билета по 350 руб.
Всего: 1050 руб.
Со скидкой 100 руб.: 950 руб.
```

Граница. Расчёт в {} не меняет переменные: после всех трёх строк price по-прежнему 350. Длинный расчёт лучше посчитать заранее в переменную — так код читается легче.

Правило: в {} пиши то, что хочешь увидеть: переменную или расчёт.

Ловушка: забыли f или скобку

main.py · без f и с f

```
name = "Лиза"
print("Привет, {name}")
print(f"Привет, {name}")
```

Вывод в консоли

```
Привет, {name}
Привет, Лиза
```

Забыли f: Python не ругается, а печатает {name} как обычный текст. Ошибку легко пропустить — смотри на вывод.

main.py · забыли }

```
name = "Лиза"
print(f"Привет, {name")
```

Вывод в консоли

```
  File ".../main.py", line 2
    print(f"Привет, {name")
                           ^
SyntaxError: f-string: expecting '}'
```

Забыли }: SyntaxError — программа не запустится вовсе. Последняя строка прямо говорит, чего не хватает: '}'.

Как правильно: f перед кавычкой и у каждой { своя }.

{price:.2f}: ровно два знака после точки

Что это. После двоеточия внутри {} пишут, *как* показать значение. .2f — два знака после точки, f значит «дробное число». **Зачем:** деньги, средний балл, проценты без «хвоста» из цифр.

main.py

```
total = 1000
friends = 3
print(f"Каждый платит {total / friends} руб.")
print(f"Каждый платит {total / friends:.2f} руб.")
```

Вывод в консоли

```
Каждый платит 333.3333333333333 руб.
Каждый платит 333.33 руб.
```

Граница. Число округляется только при выводе. В расчётах программа по-прежнему использует 333.3333…, переменная не меняется.

Правило: :.2f — вывести дробное число с двумя знаками после точки.

{views:,}: разделитель тысяч

Что это. Запятая после двоеточия делит число на тройки цифр. **Зачем:** 1500000 трудно прочитать с первого взгляда, 1,500,000 — легко. Можно вместе с .2f: :,.2f

main.py

```python
views = 1500000
print(f"Просмотров: {views}")
print(f"Просмотров: {views:,}")
balance = 1234567.891
print(f"Баланс: {balance:,.2f}")
```

Вывод в консоли

```
Просмотров: 1500000
Просмотров: 1,500,000
Баланс: 1,234,567.89
```

Граница. Python пишет числа по-английски: тысячи через запятую, дробная часть через точку. Это не ошибка, а другой стандарт записи.

Правило: :, — разделить тысячи; :,.2f — тысячи и два знака после точки.

Разбор: из чего состоит формат

{balance	:	,	.2	f}
что показать	дальше — формат	тысячи через запятую	знаков после точки	дробное число

Цифра после точки — сколько знаков оставить. Число при этом округляется:

```
main.py
speed = 3.14159
print(f"{speed:.0f}")
print(f"{speed:.1f}")
print(f"{speed:.3f}")
```

```
Вывод
3
3.1
3.142
```

.0f — без дробной части

.1f — один знак

.3f — три знака: 3.14159 округлилось до 3.142

Правило: {что : как} — слева от двоеточия значение, справа способ его показать.

Живой кодинг: «Чек магазина»

Пишем вместе по шагам

1. Спросить товар, цену (float) и количество (int).
2. Посчитать итог: цена × количество.
3. Вывести чек через f-строки.
4. Цену показать с двумя знаками, итог — с тысячами и двумя знаками.

Что должно получиться

Вывод в консоли

```
Товар: Наушники   <- ввёл пользователь
Цена: 1299.9   <- ввёл пользователь
Количество: 2   <- ввёл пользователь
Товар: Наушники
1299.90 x 2 шт.
Итого: 2,599.80 руб.
```

Стрелка — пометка для тебя, в программе её нет

Вопросы по блоку 2 • часть 1

1. Что выведет этот код? Почему программа не сообщает об ошибке?

```
total = 2599.8
print("Итого: {total:,.2f} руб.")
```

2. Что делает :.2f в записи {price:.2f}? Изменится ли после такого вывода сама переменная price?

Вопросы по блоку 2 • часть 2

3. Было speed = 3.14159 и вывод {speed:.1f}. Что будет, если поменять .1f на .0f?

4. Найди ошибку. Как её исправить?

```
total = 2599.8
print(f"Итого: {total:.2f руб.")
```

Задание: «Статистика канала»

- Программа спрашивает название канала, число подписчиков, просмотры за месяц и сколько роликов вышло за месяц.
- Выводит всё через f-строки, большие числа — с разделителем тысяч.
- Считает, сколько в среднем просмотров на один ролик, и выводит с тысячами и двумя знаками после точки.

Подсказка: среднее = просмотры / ролики. Форматы: {…:,} и {…:,.2f}. Все числа ввода — через int().

★ **Успел раньше:** посчитай доход канала, если за 1000 просмотров платят 80 руб.

Ошибка? Прочитай traceback снизу вверх и найди строку, а не меняй код наугад.

Что должно получиться

Вывод в консоли

```
Название канала: Game_Lab   <- ввёл пользователь
Подписчики: 125000   <- ввёл пользователь
Просмотры за месяц: 2340500   <- ввёл пользователь
Роликов за месяц: 12   <- ввёл пользователь
Канал: Game_Lab
Подписчики: 125,000
Просмотры: 2,340,500
В среднем на ролик: 195,041.67
```

Стрелка — пометка для тебя, в программе её нет

Проверь себя

- Ввод 125000 выводится как 125,000.
- Среднее при 2340500 и 12 роликах — 195,041.67, ровно два знака.
- В коде нет str() и склейки через +: только f-строки.

★★ Найди сам в документации: проценты

Программа спрашивает число лайков и просмотров и выводит долю лайков в процентах с одним знаком после точки.

Условие: не умножай на 100 сам. В Python есть тип формата, который делает это и ставит знак %. Найди его в документации.

Где искать: docs.python.org/ru/3/library/string.html, раздел про формат (Format Specification Mini-Language), таблица типов для дробных чисел.

Вывод в консоли

```
Лайки: 2500   <- ввёл пользователь
Просмотры: 20000   <- ввёл пользователь
Доля лайков: 12.5%
```

Стрелка — пометка для тебя, в программе её нет

УРОК 3 • БЛОК 3 ИЗ 3

Параметры print: sep и end

В этом блоке ты научишься:

- менять разделитель между значениями через sep
- менять конец строки через end и печатать несколько print в одну строку

sep: чем разделять значения

Что это. sep — разделитель, который print ставит между значениями. Если его не указать, это пробел. **Зачем:** даты, табло, списки через « | » без склейки строк.

```
print("24", "10", "2026",
```
значения через запятую

```
sep="."
```
что поставить между ними

```
)
```

main.py

```
print("Москва", "Казань", "Сочи")
print("Москва", "Казань", "Сочи", sep=" | ")
print("24", "10", "2026", sep=".")
```

Вывод в консоли

```
Москва Казань Сочи
Москва | Казань | Сочи
24.10.2026
```

Правило: sep ставится только *между* значениями, а не перед первым и не после последнего.

end: чем закончить строку

Что это. end — то, что print ставит в самом конце. Если его не указать, это переход на новую строку. Поэтому каждый print обычно начинает новую строку.

Зачем: собрать одну строку из нескольких print — например, сначала «Загрузка», а потом «готово».

main.py

```
print("Загрузка", end="...")
print("готово")
print("Следующая строка")
```

Вывод в консоли

```
Загрузка...готово
Следующая строка
```

Разбор. Первый print закончил строку на «...» вместо перехода, поэтому «готово» встало рядом. Второй print закончил как обычно — третий начал новую строку.

Правило: end меняет конец строки только у того print, где он написан.

Ловушка: end="" склеивает слова

main.py

```
print("Привет", end="")
print("мир")
print("Привет", end=" ")
print("мир")
```

Вывод в консоли

```
Приветмир
Привет мир
```

end="" — пустая строка: в конце ничего, даже пробела. Слова слиплись.

Нужен пробел — напиши его сам: end=" ".

Параметр	Если не указать	Где ставится
sep	пробел	между значениями
end	переход на новую строку	после последнего значения

Правило: sep и end ставят ровно то, что написано в кавычках, — ни символом больше.

Разбор: sep и end вместе

main.py

```
print("Раунд", 1, end=": ")
print("Игрок", "Лиза", sep="-")
print("Счёт", 3, 2, sep=":")
```

Вывод в консоли

```
Раунд 1: Игрок-Лиза
Счёт:3:2
```

print	Между значениями	Напечатал	В конце
1	пробел (sep не указан)	Раунд 1	": " — строка продолжается
2	"-"	Игрок-Лиза	переход на новую строку
3	":"	Счёт:3:2	переход на новую строку

Правило: sep действует внутри одного print, end — на стык с тем, что будет напечатано дальше.

Три способа вывода: что выбрать

main.py

```
print("Очки:", 340)
print(f"Средний балл: {4.666:.1f}")
print(10, 25, 40, sep=" | ")
```

Вывод в консоли

```
Очки: 340
Средний балл: 4.7
10 | 25 | 40
```

Способ	Когда удобно
Запятые в print	быстро вывести пару значений, формат не важен
f-строка	текст с числами, нужен формат: .2f, тысячи
sep и end	один разделитель между многими значениями или несколько print в одну строку

Правило: способы можно сочетать: f-строка внутри print с end.

Вопросы по блоку 3

1. Что выведет этот код?

```
print("Game", "Lab", sep="")
```

2. Чем print("Привет", end="") отличается от print("Привет")? Когда это заметно?

3. Найди ошибку и исправь.

```
print("24", "10", "2026", sep=.)
```

Задание: «Табло матча»

- Программа спрашивает названия двух команд и сколько голов забила каждая.
- Выводит счёт одной строкой, как на табло: «Ракеты 2 : 1 Молнии».
- Строку собирают два print: первый заканчивается на « : » через end.

Подсказка: голы можно не превращать в int — с ними ничего не считаем. Пробелы вокруг двоеточия входят в end: " : ".

★ **Успел раньше:** спроси минуту матча и выведи её в той же строке: «Ракеты 2 : 1 Молнии 78'».

Что должно получиться

Вывод в консоли

```
Команда 1: Ракеты    <- ввёл пользователь
Команда 2: Молнии    <- ввёл пользователь
Голы команды 1: 2    <- ввёл пользователь
Голы команды 2: 1    <- ввёл пользователь
Ракеты 2 : 1 Молнии
```

Стрелка — пометка для тебя, в программе её нет

Проверь себя

- Счёт напечатан одной строкой, хотя print два.
- Вокруг двоеточия ровно по одному пробелу.
- Можешь объяснить, что делает end в первом print.

★★ Поле для «Крестиков-ноликов»

Программа спрашивает твой символ и символ соперника и рисует поле с готовой позицией, как на примере.

Клетки строки разделены « | », между строками — линия. Пустая клетка — пробел.

Условие: ровно 5 print и никакой склейки через + — только sep. Придумай, как хранить пустую клетку.

Вывод в консоли

```
Твой символ: X   <- ввёл пользователь
Символ соперника: O   <- ввёл пользователь
X | O | X
--+---+--
  | X | O
--+---+--
O |   | X
```

Стрелка — пометка для тебя, в программе её нет

Это пригодится в Проекте 2 «Игра на поле».

Вопросы по всей теме

1. В «Чеке магазина» убрали int(): count = input("Количество: "). Что случится в строке total = price * count и почему?

2. Что выведет этот код? Что изменится, если убрать end=" "?

```
price = 99.5
print(f"Цена: {price:.0f}", end=" ")
print("руб.", "без скидки", sep="/")
```

3. Тема 1.1: почему в «Фокусе с числом» вышло 5.0, а не 5? Как напечатать 5 без «.0» тем, что мы узнали сегодня?

Мини-проект: «Касса кинотеатра»

Программа работает как касса: спрашивает фильм, время сеанса, сколько билетов и почём, а потом печатает аккуратный билет с итогом.

В ней сойдутся все три блока: ввод и числа, f-строки, sep и end.

Это пригодится в Проекте 3 «Мультитул»: такой чек станет одним из его инструментов.

Вывод в консоли

```
Фильм: Космический рейс   <- ввёл пользователь
Сеанс, час: 19   <- ввёл пользователь
Сеанс, минуты: 30   <- ввёл пользователь
Сколько билетов: 3   <- ввёл пользователь
Цена билета: 450   <- ввёл пользователь
------ БИЛЕТЫ ------
Фильм: Космический рейс
Сеанс: 19:30
3 x 450.00 руб.
Итого: 1,350.00 руб.
```

Стрелка — пометка для тебя, в программе её нет

Что должна уметь касса

1. Спрашивает фильм, время сеанса, сколько билетов и сколько стоит один билет.
2. Считает, сколько заплатить за все билеты.
3. Печатает аккуратный билет: время как 19:30, цену с двумя знаками, итог с разделителем тысяч.

План по шагам

Шаг	Что сделать	Тема	Как проверить
1	Спросить фильм, час и минуты сеанса	1.2, блок 1	Запусти — программа задаёт 3 вопроса
2	Спросить число билетов (int) и цену (float), посчитать total	1.1, 1.2	Временный print(total) при 3 и 450 выводит 1350.0
3	Шапка, фильм и строка «3 x 450.00 руб.» через f-строки	1.2, блок 2	Цена выводится с двумя знаками
4	Время через end и sep=":", итог через :,.2f	1.2, блоки 2–3	Видно «Сеанс: 19:30» и «Итого: 1,350.00 руб.»

Проверяй после каждого шага, а не в самом конце. Застрял — 15 минут ищешь сам, потом спрашиваешь.

Заготовка: заполни пропуски

main.py

```
# Касса кинотеатра
# Шаг 1: спроси фильм, час и минуты сеанса
film = input("Фильм: ")
# hour = ...
# minute = ...
# Шаг 2: спроси число билетов (int) и цену (float)
# tickets = ...
# price = ...
# total = ...
# Шаг 3: шапка и фильм через f-строки
print("------ БИЛЕТЫ ------")
# Шаг 4: время через end и sep=":", потом цена и итог
```

Строки с # — подсказки. Убери # и допиши код вместо «...».

Заготовка уже запускается: спрашивает фильм и печатает шапку.

Каждую строку своего кода ты должен уметь объяснить.

Готово, если...

- **Блок 1:** при вводе 3 билетов по 450 программа не падает; при вводе «три» — ValueError, и ты можешь объяснить почему.
- **Блок 2:** цена выводится как 450.00, итог — как 1,350.00.
- **Блок 3:** время выводится одной строкой: «Сеанс: 19:30».

★ Попкорн

Спроси, сколько попкорна по 190 руб., и добавь в итог.

```
Попкорн: 2 x 190 руб.
Итого: 1,730.00 руб.
```

★★ Школьные билеты

Спроси, сколько билетов школьные: они на 30% дешевле. Покажи цену, итог и экономию.

```
Школьный билет: 315.00 руб.
Итого: 1,080.00 руб.
Экономия: 270.00 руб.
```

Челлендж: «Сколько это в год?»

Программа спрашивает, сколько минут в день ты играешь или слушаешь музыку, и считает: часы в неделю, часы в год, сколько это суток без перерыва и какая это доля года. Каждое число — с одним знаком после точки.

Вывод в консоли

```
Сколько минут в день ты играешь или слушаешь музыку: 90   <- ввёл пользователь
В неделю: 10.5 ч
В год: 547.5 ч
Это 22.8 суток без перерыва
Доля года: 6.2%
```

Стрелка — пометка для тебя, в программе её нет

Придумай сам, как посчитать каждую строку. Долю года — через формат процентов, если нашёл его в документации, или умножив на 100.

ИТОГИ ТЕМЫ 1.2

Теперь я умею

- получать ответы пользователя через input() и помнить, что это строка
- превращать ввод в числа через int() и float(), а числа в текст — через str()
- проверять, что введены только цифры, через .isdigit()
- читать traceback снизу вверх и находить, где и почему упала программа
- вставлять значения и расчёты в текст через f-строки
- выводить числа с нужным числом знаков и разделителем тысяч
- собирать вывод через sep и end